

Proyecto de *Entornos de Desarrollo* y de *Base de Datos*

DISEÑO E IMPLEMENTACIÓN DE UNA BASE DE DATOS



Juan Manuel Jiménez Olmedo

ÍNDICE:



1. Problema planteado y objetivos de proyecto
2. MERISE
 - 2.1. Fase de Análisis Conceptual (Modelo Conceptual de Datos - MCD)
 - 2.2. Fase de Diseño Lógico (Modelo Lógico de Datos - MLD)
 - 2.3. Fase de Diseño Físico (Modelo Físico de Datos - MPD)
3. Conexión con la base datos y eliminación de las tablas existentes
4. Creación de tablas y descarga e inserción de datos
5. Pruebas unitarias
6. Conclusiones



1. Problema planteado y objetivos de proyecto

☐ Situación inicial:

- Inexistencia de un sistema centralizado de consulta estadística.
- Dificultad de análisis y seguimiento.
- Ausencia de métodos rápidos y precisos en la obtención de información.



1. Problema planteado y objetivos de proyecto

□ Objetivos:

- Optimización del proceso de búsqueda y registro estadístico.
- Manutención de la integridad de la información.
- Mejora en la trazabilidad de las estadísticas.

2. MERISE



☐ Análisis + Diseño + Gestión

☐ Tres niveles:

- Análisis conceptual: Entidades principales y sus relaciones.
- Diseño Lógico: Definición de tablas y atributos.
- Diseño Físico: Definición de datos y creación de restricciones.



2. MERISE: Fase de Análisis Conceptual

□ Entidades principales:

- **Conference:** Conferencias 'east' y 'west' junto con sus divisiones.
- **Team:** Equipos vinculados a una conferencia.
- **Player:** Jugadores vinculados a un equipo.
- **Stadium:** Estadios donde se juegan los partidos.
- **Fault:** Tipos de falta en un partido (técnica, personal o antideportiva).
- **Season:** Temporada del torneo.
- **Game:** Partidos realizados, vinculados a un estadio y a una temporada.
- **Game_Team:** Relación entre partidos y equipos, incluyendo ganador y puntos anotados.
- **Player_Fault:** Registro de faltas cometidas por jugadores en partidos específicos.
- **Team_Position:** Posición final de un equipo en una temporada.
- **Player_Point:** Puntos anotados por un jugador en un juego (triples, dobles o tiros libres).



2. MERISE: Fase de Análisis Conceptual

☐ Diagrama conceptual

☐ Relaciones identificadas:

- Un equipo pertenece a una conferencia (uno a muchos).
- Un jugador pertenece a un equipo (uno a muchos).
- Un juego se realiza en un estadio y en una temporada (uno a muchos).
- Un juego involucra a varios equipos (muchos a muchos) a través de ***Game_Team***.
- Un jugador puede cometer varias faltas en varios juegos (muchos a muchos) a través de ***Player_Fault***.
- Un equipo tiene una posición por temporada (uno a muchos).
- Un jugador puede anotar puntos en varios juegos en distintas temporadas (muchos a muchos) a través de ***Player_Point***.



2. MERISE: Fase de Diseño Lógico

□ Tablas y atributos:

- **Conference** (id, name, division_name)
- **Team** (id, name, conference_id)
- **Player** (id, team_id, name)
- **Stadium** (id, name, city)
- **Fault** (id, name)
- **Season** (id, year, name)
- **Game** (id, date, stadium_id, season_id)
- **Game_Team** (id, game_id, team_id, winner, points)
- **Player_Fault** (id, player_id, game_id, fault_id)
- **Team_Position** (id, team_id, position, season_id)
- **Player_Point** (id, player_id, game_id, three_point, two_point, free_point, season_id)



2. MERISE: Fase de Diseño Lógico

☐ Claves primarias:

- Cada tabla tiene la clave primaria única *id*

☐ Claves foráneas:

- | | |
|-----------------|-------------|
| • conference_id | • season_id |
| • team_id | • fault_id |
| • game_id | • player_id |

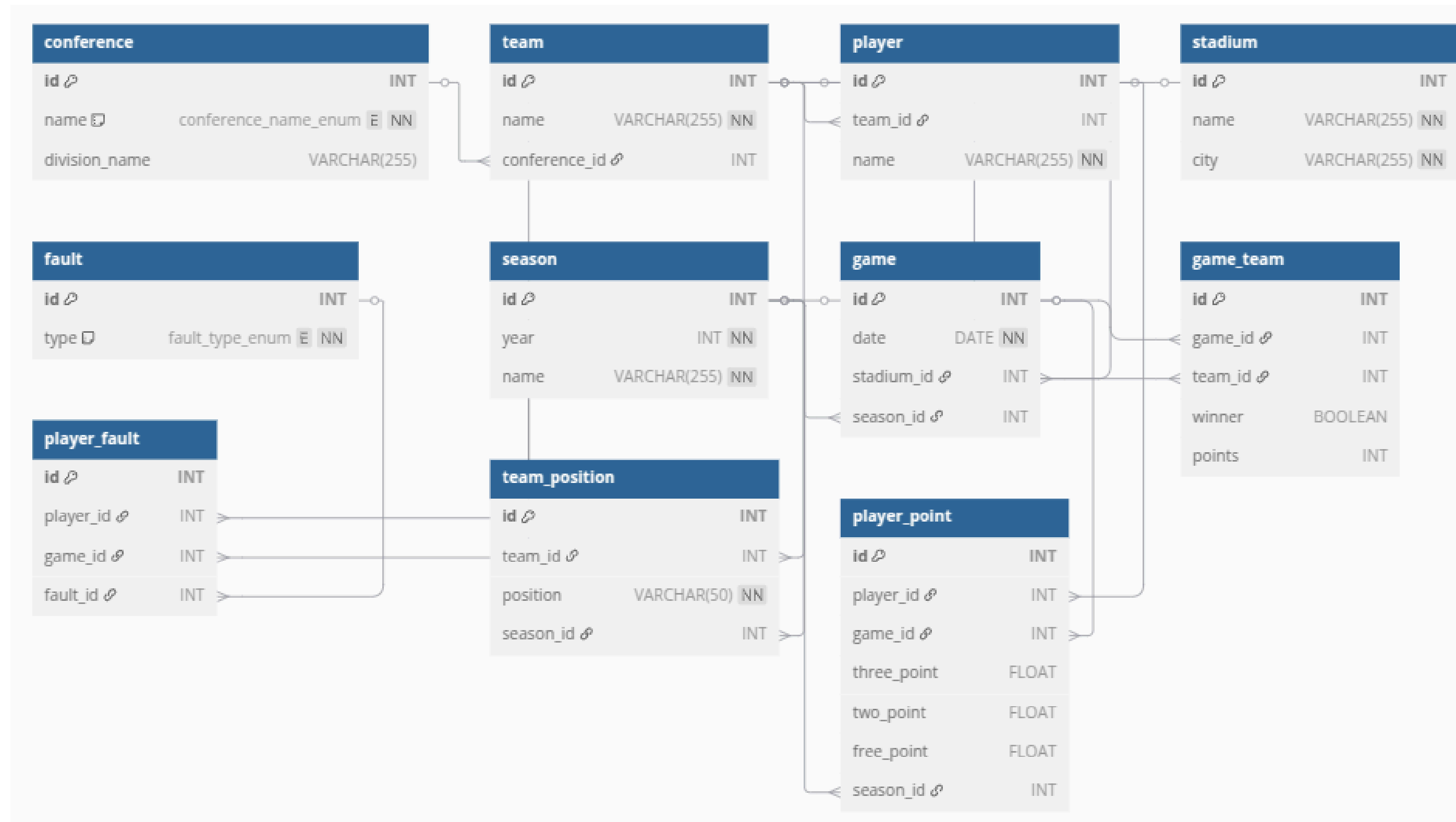


2. MERISE: Fase de Diseño Físico

- ☐ Tipos de datos: *INT, VARCHAR, ENUM, DATE* y *FLOAT*.
- ☐ Restricciones de integridad: *NOT NULL, AUTO_INCREMENT*, claves primarias y foráneas.
- ☐ Índices (como *game_id, team_id* o *player_id*).
- ☐ Reglas de integridad referencial (mediante claves foráneas).



2. MERISE: Fase de Diseño Físico



3. Conexión con la BB.DD. y eliminación de tablas existentes



□ Conexión con la BB.DD:

1) Instalar MySQL en Google Colab: `!pip install mysql-connector-python`

2) Establecer conexión:

```
import mysql.connector

conexion = mysql.connector.connect(
    user="dav_gal",
    password="xxx-xxx-xxx",
    host="82.165.173.36",
    port="31234"
)

print("Conexión exitosa a la base de datos.")
```



3. Conexión con la BB.DD. y eliminación de tablas existentes

□ Eliminación de las tablas existentes:

3) Eliminar las tablas existentes antes de crear unas nuevas:

```
cursor = conexion.cursor()
cursor.execute("SET foreign_key_checks = 0;")
cursor.execute("""
    DROP TABLE IF EXISTS conference, team, player, stadium, fault,
    season, game, game_team, player_fault, team_position, player_point;
""")

cursor.execute("SET foreign_key_checks = 1;")

cursor.execute("SHOW TABLES")
tables = cursor.fetchall()
print(tables)
```




4. Creación de tablas y descarga e inserción de datos

□ Ejemplos de creación de tablas:

- *conference* (Conferencias):

```
cursor.execute(
    """
    CREATE TABLE conference (
        id INT PRIMARY KEY AUTO_INCREMENT,
        name ENUM ('east', 'west') NOT NULL,
        division_name VARCHAR(255)
    );
    """)
```

- *team* (Equipos):

```
cursor.execute(
    """
    CREATE TABLE team (
        id INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(255) NOT NULL,
        conference_id INT,
        FOREIGN KEY (conference_id)
        REFERENCES conference (id)
    );
    """)
```



4. Creación de tablas y descarga e inserción de datos

□ Descarga de datos desde el repositorio en GitHub:

```
!git clone https://github.com/DavidGallardoDigi/proyectoDbNba.git  
import sys  
sys.path.append("/content/proyectoDbNba")
```

□ Ejemplo de inserción de datos en una tabla:

```
from data_conference import data_insert_conference  
cursor.execute(data_insert_conference)  
conexion.commit()
```



5. Pruebas unitarias

□ Conexión con la BB.DD:

1) Instalar MySQL en Google Colab: `!pip install mysql-connector-python`

2) Establecer conexión:

```
import mysql.connector
import unittest

class DatabaseTestCase(unittest.TestCase):
    """Clase base para pruebas de la base de datos universitaria."""

    @classmethod
    def setUpClass(cls):
        cls.conn = mysql.connector.connect(
            user="dav_gal",
            password="xxx-xxx-xxx",
```

(...)

```
(...)
        host="82.165.173.36",
        port="31234",
        database="damL_proyecto2T_g2"
    )
    cls.cursor = cls.conn.cursor()

    @classmethod
    def tearDownClass(cls):
        cls.cursor.close()
        cls.conn.close()
```




5. Pruebas unitarias

□ *TestTables* (existencia de tablas):

- Comprueba que la BB.DD. contiene las tablas esperadas:

```
class TestTables(DatabaseTestCase):
    """Prueba la existencia de las tablas en la base de datos."""
    def test_tables_exist(self):
        self.cursor.execute("SHOW TABLES")
        tables = {table[0] for table in self.cursor.fetchall()}
        expected_tables = {
            "conference", "team", "player", "stadium", "fault", "season",
            "game", "game_team", "player_fault", "team_position", "player_point"
        }
        self.assertEqual(tables, expected_tables, "Las tablas existentes no coinciden con las esperadas")

print("\n=== EJECUCIÓN: TestTables ===\n")
suite_tables = unittest.TestLoader().loadTestsFromTestCase(TestTables)
result_tables = unittest.TextTestRunner(verbosity=2).run(suite_tables)
```



5. Pruebas unitarias

□ *TestCounts* (conteo de registros):

- Verifica que cada tabla posea el número de registros esperado.
- Ejemplo con *fault*:

```
class TestCounts(DatabaseTestCase):
    """Verifica que cada tabla tenga la cantidad de registros esperada."""
    (...)
    def test_fault_count(self):
        self.cursor.execute("SELECT COUNT(*) FROM fault")
        count = self.cursor.fetchone()[0]
        self.assertEqual(count, 3, "La tabla fault debería tener 3 registros")
    (...)
print("\n=== EJECUCIÓN: TestCounts ===\n")
suite_counts = unittest.TestLoader().loadTestsFromTestCase(TestCounts)
result_counts = unittest.TextTestRunner(verbosity=2).run(suite_counts)
```



5. Pruebas unitarias

□ *TestQueries* (consultas específicas):

- Muestra si determinadas consultas devuelven resultados acertados.
- Ejemplo con *players* y *teams* (lista de jugadores con sus equipos):

```
class TestQueries(DatabaseTestCase):
    """Verifica que las consultas SQL devuelvan los resultados esperados."""

    def test_players_and_teams(self):
        """Obtiene la lista de jugadores junto con sus equipos."""
        query = """
            SELECT p.id, p.name, t.name
            FROM player p
            INNER JOIN team t ON p.team_id = t.id;
        """
        self.cursor.execute(query)
        result = self.cursor.fetchall()
        self.assertGreaterEqual(len(result), 1, "La consulta de jugadores y equipos no devolvió resultados"
        (...)
```

```
(...)
print("\n=== EJECUCIÓN: TestQueries ===\n")
suite_queries = unittest.TestLoader().loadTestsFromTestCase(TestQueries)
result_queries = unittest.TextTestRunner(verbosity=2).run(suite_queries)
```



5. Pruebas unitarias

□ *TestBooleanAssertions* (validaciones lógicas):

- Verifican ciertos *statements* irrefutables por su naturaleza.
- Ejemplo: “No debe haber equipos sin conferencia”:

```
class TestBooleanAssertions(DatabaseTestCase):
    """Ejemplos de uso de assertTrue y assertFalse en consultas a la BD."""

    def test_no_team_without_conference(self):
        """Verifica que no existan equipos sin conferencia asignada (conference_id)."""
        query = "SELECT id FROM team WHERE conference_id IS NULL"
        self.cursor.execute(query)
        rows = self.cursor.fetchall()
        self.assertTrue(len(rows) == 0, "Se encontraron equipos sin conferencia asignada")

(...)

print("\n=== EJECUCIÓN: TestQueries ===\n")
suite_queries = unittest.TestLoader().loadTestsFromTestCase(TestQueries)
result_queries = unittest.TextTestRunner(verbosity=2).run(suite_queries)
```



5. Pruebas unitarias

□ *TestReferentialIntegrity* (integridad referencial):

- Comprueba que *game_team* no tenga claves foráneas huérfanas:

```
class TestReferentialIntegrity(DatabaseTestCase):
    """Comprueba que no existan registros huérfanos en game_team."""

    def test_no_orphan_game_team(self):
        """
        Verifica que la tabla game_team no tenga 'game_id' o 'team_id'
        apuntando a registros inexistentes en las tablas 'game' o 'team'.
        """
        query = """
            SELECT gt.id
            FROM game_team gt
            LEFT JOIN game g ON gt.game_id = g.id
```

(...)

(...)

```
        LEFT JOIN team t ON gt.team_id = t.id
        WHERE g.id IS NULL
        OR t.id IS NULL
        """
        self.cursor.execute(query)
        orphans = self.cursor.fetchall()
        self.assertTrue(
            len(orphans) == 0,
            f"Se encontraron filas huérfanas en game_team:
{orphans}"
        )
```


5. Pruebas unitarias

□ *Prueba de rendimiento:*

- Mide el rendimiento de una consulta cien veces.
- Resultados obtenidos:
 - 1) Tiempo total para 100 ejecuciones: 11.8051 segundos.
 - 2) Tiempo promedio por ejecución: 0.118051 segundos.

```
import timeit
```

```
# Creamos una conexión separada para las pruebas de rendimiento
```

```
perf_conn = mysql.connector.connect(  
    user="dav_gal",  
    password="xxx-xxx-xxx",  
    host="82.165.173.36",  
    port="31234",  
    database="damL_proyecto2T_g2"  
)
```

```
perf_cursor = perf_conn.cursor()
```

```
def perf_query_problem1():
```

```
    """Ejecuta la consulta de rendimiento para game_team y retorna los resultados."""
```

```
    query = """
```

```
        SELECT g.id, g.date, t.name
```

```
        FROM game g
```

```
        INNER JOIN game_team gt ON g.id = gt.game_id
```

```
        INNER JOIN team t ON gt.team_id = t.id;
```

```
    """
```

```
    perf_cursor.execute(query)
```

```
    return perf_cursor.fetchall()
```

```
# Medimos el tiempo de ejecución de la consulta 100 veces
```

```
execution_time = timeit.timeit("perf_query_problem1()", globals=globals(),  
                                number=100)
```

```
print(f"Tiempo total para 100 ejecuciones: {execution_time:.4f} segundos")
```

```
print(f"Tiempo promedio por ejecución: {execution_time/100:.6f} segundos")
```

```
# Cerramos la conexión de rendimiento
```

```
perf_cursor.close()
```

```
perf_conn.close()
```





6. Conclusiones

- ☐ La estructura de la BB.DD. es coherente con la propuesta de proyecto.
- ☐ Precisión, consistencia e integridad de los datos utilizados.
- ☐ Funcionamiento correcto de las consultas realizadas.
- ☐ Desempeño eficiente en el procesamiento de información.